CS543 Web Services – Assignment 3

Integrate an External SOAP Partner – CampusEats

Team ID	17
Roll No / Name	1. Anurag Tiwari (20251651028)
	2. Mohd Rashid (20251651057)
	3. Saurabh Singh (20251651089)
	4. Niraj Kumar Gupta (20251651066)

1. Context

Partner: PayFast Payment Gateway (external payment processor)

Operation integrated: charge – authorizes and captures a payment for a CampusEats order.

This edge is modeled as SOAP, while the rest of CampusEats (student-facing ordering, menu browsing, order tracking) stays REST, for three reasons. First, a payment call needs a strong, machine-checkable contract: the WSDL fixes the exact request and response shape so a mismatch is caught at build time rather than discovered from a malformed JSON body in production. Second, the transaction touches money, so it needs message-level guarantees – a signed/encrypted SOAP header can protect the credentials and the payload independently of the transport, whereas plain REST over HTTPS only protects the message in transit. Third, payment gateways of this era were themselves built and documented as WSDL services with well-defined fault contracts (declined, expired, gateway-unavailable), so consuming them as SOAP avoids re-inventing an ad-hoc REST wrapper around a partner that was never designed to speak REST.

2. HTTP Binding

The SOAP request in **soap-request.xml** is carried as an HTTP POST to the endpoint declared in the WSDL's `service/port` element, using the `soapAction` value declared in the binding.

```
POST /payfast/v1/charge HTTP/1.1
Host: api.payfast-gateway.example.com
Content-Type: text/xml; charset=utf-8
Content-Length: 612
SOAPAction: "http://campuseats.example.com/payfast/charge"

<soap:Envelope ...>
  ... (see soap-request.xml for the full body)
</soap:Envelope>
```

3. Discovery

CampusEats obtains this contract in the modern, UDDI-free way: PayFast publishes its WSDL at a fixed, versioned URL (`https://api.payfast-gateway.example.com/payfast?wsdl`), which the CampusEats backend team downloads once at integration time and checks into the repository as `partner.wsdl`, pinned to a version. There is no live UDDI registry lookup at runtime; instead CampusEats keeps its own internal partner catalogue – a simple table (or a services.yaml / registry

service) that records every external partner it integrates, acting as a tModel-style pointer from "partner name" to "where its contract and endpoint live".

Field	Value
Business	PayFast Payment Gateway
Service	PayFast Charge Service
Endpoint	https://api.payfast-gateway.example.com/payfast/v1/charge
WSDL (tModel pointer)	https://api.payfast-gateway.example.com/payfast?wsdl

4. Fault Mapping

PayFast's fault vocabulary is never exposed to CampusEats students; it is caught at the integration boundary and translated into the error contract that `placeOrder` already promised in Assignment 2.

Partner fault (PayFast)	CampusEats placeOrder error
card_declined	PAYMENT_DECLINED – "Payment declined, try another card."
gateway_unavailable / timeout	PAYMENT_GATEWAY_UNAVAILABLE – "Payment service temporarily u

The adapter layer that calls PayFast catches the `soap:Fault` shown in `soap-fault.xml`, reads the partner's `code` element, and raises CampusEats' own domain exception so the rest of the placeOrder flow only ever sees CampusEats-shaped errors.

5. Files Submitted

• integration.pdf – this document (context, HTTP binding, discovery, fault mapping).

• partner.wsdl – the PayFast partner contract.

• soap-request.xml – the charge request envelope.

• soap-response.xml – the success response envelope.

• soap-fault.xml – the card_declined fault envelope.